

## 1. I/O HARDWARE ARCHITECTURE &amp; COMMUNICATION MECHANISMS

## Device Controller &amp; Registers

Electronic component on motherboard/expansion card bridging device and system bus:

- **Data-In Register:** Host reads data received from device.
- **Data-Out Register:** Host writes data to send to device.
- **Status Register:** Bits for *Busy*, *Ready*, *Error* flags.
- **Control Register:** Host writes commands (Read, Write, Seek).

**Addressing:** PMIO (Port I/O: special in/out CPU instructions) vs MMIO (Memory-Mapped I/O: registers mapped to RAM address space, accessed via standard pointers).

## Polling vs Interrupt-Driven I/O

- **Polling (Busy-Waiting):** CPU repeatedly reads Status Register *Busy* bit in tight loop. Simple; acceptable for ultra-fast devices, but totally starves CPU on slow devices.
- **Interrupt-Driven I/O:** CPU initiates I/O and switches to another process. Device controller asserts Interrupt Request Line (IRL) when ready. CPU halts current instruction, saves context, indexes **Interrupt Vector Table (IVT)**, and invokes **Interrupt Service Routine (ISR)**.
- **Interrupt Hardware:** Non-Maskable (NMI: fatal hardware faults) vs Maskable (can be delayed during critical sections).

## Direct Memory Access (DMA)

Specialized hardware bypasses CPU for high-speed block transfers (Disks, NICs):

- **Architecture:** CPU programs DMAC (DMA Controller) with Source Address, Destination Address, Transfer Count, and Command (R/W), then yields bus control.
- **Execution:** DMAC transfers entire data blocks directly between device buffer and physical RAM.
- **Cycle Stealing:** DMAC snatches memory bus cycles from CPU while CPU computes internally from L1/L2 cache.
- **Completion:** DMAC generates exactly **1 interrupt per block** (instead of 1 interrupt per byte).

## 2. KERNEL I/O SUBSYSTEM SERVICES &amp; ABSTRACTIONS

## I/O Scheduling, Buffering &amp; Caching

- **I/O Scheduling:** OS reorders pending request queues to optimize physical device efficiency, minimize seek latency, and ensure fairness across processes.
- **Buffering:** Memory area holding data during transfer to resolve:
  - *Speed Mismatch:* Fast CPU vs slow modem/disk.
  - *Transfer Size Mismatch:* Network packets vs disk block chunks.
  - *Copy Semantics:* Preserving application data state at write() time.
  - *Schemes:* Single Buffering, Double Buffering (Ping-Pong: CPU processes buf 1 while device fills buf 2), Circular Buffering.
- **Caching vs Buffering:** Caching keeps copies of frequently read data in fast RAM; Buffering temporarily holds transit data between producer and consumer.

## Spooling, Error Handling &amp; Device Reservation

- **Spooling (Simultaneous Peripheral Operations On-Line):** Disk buffer holding output jobs for non-sharable / non-multiplexable devices (e.g. Printer, Tape drives). OS printer daemon consumes spool queue sequentially while user processes finish immediately without blocking.
- **Error Handling:** Transient errors (parity faults, transient seek errors) auto-retried with exponential backoff. Permanent errors propagated via system error codes (errno: EIO, ENOSPC).
- **I/O Protection:** All I/O instructions are privileged (Kernel Mode only). User applications invoke I/O strictly via System Calls (Trap to Kernel).
- **Device Streams:** UNIX STREAMS architecture enables modular pipelining of device drivers, protocol stacks, and line disciplines.

## 3. PHYSICAL MAGNETIC DISK GEOMETRY &amp; ACCESS LATENCY EQUATIONS

## Physical Architecture &amp; Terminology

- **Platters & Spindle:** Stack of circular aluminum/glass platters coated with magnetic material rotating on a common spindle at constant angular velocity (5400, 7200, 10000, 15000 RPM).
- **Tracks & Sectors:** Each surface is divided into concentric rings called **Tracks**. Each track is partitioned into **Sectors** (standard 512 Bytes or Advanced Format 4 KB).
- **Cylinders:** Set of all tracks at identical radial distance across all platter surfaces. Data across a cylinder can be accessed without moving the mechanical arm!
- **Read/Write Heads:** Aerodynamically float on an air cushion (~nanometers above platter) mounted on a synchronized movable actuator arm.

## Access Time Formulas &amp; Solved Numerical

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotational}} + T_{\text{transfer}} + T_{\text{controller}}$$

- **Seek Time ( $T_{\text{seek}}$ ):** Time to move actuator arm to target cylinder (Dominant mechanical bottleneck: 3 - 9 ms).
- **Rotational Latency ( $T_{\text{rotational}}$ ):** Time for target sector to rotate under head.
 
$$T_{\text{rot}}(\text{avg}) = 60 / (2 * \text{RPM}) \text{ sec} = 30000 / \text{RPM ms}$$
 For 7200 RPM:  $T_{\text{rot}}(\text{avg}) = 30,000 / 7200 = 4.167 \text{ ms}$ .
- **Transfer Time:**  $T_{\text{trans}} = (\text{Bytes to transfer}) / (\text{Transfer Rate})$ .
- **Controller Overhead:** Handshake & bus protocol delay (~0.1 - 0.5 ms).

**AKTU Numerical:** 7200 RPM, avg seek 6 ms, transfer rate 50 MB/s. Time to read 512 B sector?

$$T = 6 \text{ ms (seek)} + 4.167 \text{ ms (rot)} + (512 / 50 * 10^6 = 0.01 \text{ ms}) + 0.2 \text{ ms} = 10.377 \text{ ms}$$

1. DISK SCHEDULING ALGORITHMS MASTER BENCHMARK & COMPARISON

Standard Benchmark Parameters (AKTU PYQ): Request Queue = 98, 183, 37, 122, 14, 124, 65, 67 | Initial Head = 53 | Disk Range = 0 to 199 | Direction = Towards Higher Cylinders

| Algorithm               | Head Movement Trace Path                            | Seek Count     | Core Pros                                           | Core Cons                                            |
|-------------------------|-----------------------------------------------------|----------------|-----------------------------------------------------|------------------------------------------------------|
| FCFS<br>First-Come      | 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67       | 640 cyl        | 100% Fair, Zero overhead, No starvation             | Disastrous mechanical wild swings; worst seek count  |
| SSTF<br>Shortest Seek   | 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183       | 236 cyl        | High throughput, greedily minimizes seek time       | Starvation of edge/distant requests under heavy load |
| SCAN<br>Elevator        | 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14 | 236 cyl        | Prevents starvation, uniform mechanical sweep       | Always travels to disk boundary (199); favors middle |
| C-SCAN<br>Circular SCAN | 53 → ... → 183 → 199 → 0 (no service) → 14 → 37     | 382 cyl        | Uniform wait time across all cylinders              | Wastes seek time on full return sweep (199 → 0)      |
| LOOK<br>Boundary Opt.   | 53 → 65 → 67 → 98 → 122 → 124 → 183 → 37 → 14       | 208 cyl (Best) | Optimal! Reverses at last request; skips boundaries | Direction state tracking overhead in kernel          |
| C-LOOK<br>Circular LOOK | 53 → ... → 183 → 14 (jumps to lowest request) → 37  | 322 cyl        | Uniform waiting without traversing dead ends        | Higher seek than LOOK due to return hop              |

2. RAID ARCHITECTURE (REDUNDANT ARRAY OF INDEPENDENT DISKS)

**1. Striping (Performance)**

Data split into consecutive chunks/stripes spread across multiple disks in parallel. Increases read/write throughput via multi-channel parallelism. Provides 0 fault tolerance.

**2. Mirroring (Redundancy)**

Identical copy of data written simultaneously to duplicate mirror disk. 100% data protection against single disk failure; fast read load balancing. 50% storage overhead.

**3. Parity (Error Recovery)**

Bitwise XOR error-correcting code ( $P = D1 \oplus D2 \oplus D3$ ). If any disk fails:  $D1 = P \oplus D2 \oplus D3$ . Saves storage vs mirroring.

| RAID Level    | Architecture & Parity Strategy               | Usable Capacity              | Min Disks | Fault Tolerance     | Write Penalty & Primary Use Case                                  |
|---------------|----------------------------------------------|------------------------------|-----------|---------------------|-------------------------------------------------------------------|
| RAID 0        | Block-level Striping without redundancy      | $N \times S$ (100%)          | 2         | 0 disks (Data loss) | 0 penalty. High-speed video editing, scratch disks.               |
| RAID 1        | Disk Mirroring (Shadowing)                   | $1 \times S$ (50% if $N=2$ ) | 2         | $N - 1$ disks       | 2× writes. OS system boot drives, critical DB logs.               |
| RAID 2        | Bit-level Striping + Hamming Code ECC        | $(N - ECC) \times S$         | 3         | 1 disk              | Heavy controller ECC overhead. Obsolete.                          |
| RAID 3        | Byte-level Striping + Dedicated Parity disk  | $(N - 1) \times S$           | 3         | 1 disk              | Dedicated parity disk is heavy I/O bottleneck.                    |
| RAID 4        | Block-level Striping + Dedicated Parity disk | $(N - 1) \times S$           | 3         | 1 disk              | Parity disk bottleneck on small concurrent writes.                |
| RAID 5        | Block-level Striping + Distributed Parity    | $(N - 1) \times S$           | 3         | 1 disk failure      | 4 I/Os write penalty (2 reads + 2 writes). Enterprise servers.    |
| RAID 6        | Dual Distributed Parity (P + Q Reed-Solomon) | $(N - 2) \times S$           | 4         | 2 disks failure     | 6 I/Os write penalty (3 reads + 3 writes). Large archival arrays. |
| RAID 10 (1+0) | Striped set of Mirrored pairs                | $(N / 2) \times S$           | 4         | 1 per mirrored pair | High performance & faster rebuild than RAID 0+1. Production DBs.  |

RAID 5 Write Penalty Derivation: Updating 1 block requires: Read old data, Read old parity, Compute new parity ( $P_{new} = P_{old} \oplus D_{old} \oplus D_{new}$ ), Write new data, Write new parity = 2 Reads + 2 Writes = 4 Disk I/O operations.

1. FILE CONCEPT, DIRECTORY SYSTEMS & VIRTUAL FILE SYSTEM (VFS)

**File Control Block (FCB) & Directories**

- FCB / Inode:** File metadata container: Identifier, Type, Size, Permissions (rwx), Timestamps (atime, mtime, ctime), Owner ID, and Disk Block Pointers. Does NOT store file name in UNIX!
- Directory Structures:** Single-Level (name collision), Two-Level (user isolation), Hierarchical Tree (paths /a/b/c), Acyclic Graph (shared files via links).
- Hard Link:** New directory entry pointing to *existing Inode*. Increments i\_nlink. Deleting original preserves data. Cannot cross filesystems.
- Soft (Symbolic) Link:** New file with *distinct Inode* containing destination path text string. Can cross filesystems; creates dangling link if target deleted.

**Virtual File System (VFS) Architecture**

Kernel abstraction layer decoupling generic POSIX system calls (open, read, write) from concrete filesystem implementations (ext4, NTFS, NFS, FAT):

- Superblock Object:** Entire mounted filesystem metadata (magic number, block size, total blocks, free blocks).
- Inode Object:** Individual file physical metadata and block pointers.
- Dentry Object:** Directory entry mapping human-readable string path names to Inode numbers; cached in fast RAM **dcache**.
- File Object:** Per-process open file instance (file offset pointer, access mode flags).

2. FILE ALLOCATION METHODS & UNIX INODE MATHEMATICAL DERIVATION

| Method     | Block Layout Scheme                       | Access Performance                       | Fragmentation                 | Key Trade-off                                                   |
|------------|-------------------------------------------|------------------------------------------|-------------------------------|-----------------------------------------------------------------|
| Contiguous | Sequential contiguous disk blocks         | Fast Sequential & <b>O(1) Direct</b>     | <b>External Fragmentation</b> | File size cannot grow dynamically without relocation            |
| Linked     | Each block stores pointer to next block   | Sequential only; <b>O(N) Direct</b>      | Zero External; internal only  | Slow seek traversal; 1 corrupted pointer destroys rest of file  |
| FAT        | Pointers stored in dedicated RAM table    | Fast Linked traversal in RAM             | Zero External                 | Entire FAT table must reside in physical RAM (memory cost)      |
| Indexed    | Index block stores pointers to all blocks | <b>O(1) Direct &amp; Fast Sequential</b> | Zero External                 | Index block overhead for small files; requires multilevel index |

**UNIX Inode Sizing Math (AKTU 10-Marks)**

Inode has 12 Direct, 1 Single, 1 Double, 1 Triple Indirect pointer.

**Pointers/Block  $K = \text{Block\_Size} / \text{Pointer\_Size}$**

Given: Block = 4 KB, Pointer = 4 B  $\Rightarrow K = 4096 / 4 = 1024 (2^{10})$

- Direct:  $12 \times 4 \text{ KB} = \mathbf{48 \text{ KB}}$
- Single Indirect:  $1 \times 1024 \times 4 \text{ KB} = \mathbf{4 \text{ MB}}$
- Double Indirect:  $1 \times (1024)^2 \times 4 \text{ KB} = \mathbf{4 \text{ GB}}$
- Triple Indirect:  $1 \times (1024)^3 \times 4 \text{ KB} = \mathbf{4 \text{ TB}}$

**Max File Size = 48 KB + 4 MB + 4 GB + 4 TB  $\approx$  4 TB**

**Free Space Management Schemes**

- Bit Vector / Bitmap:** 1 bit per block (0=Alloc, 1=Free). Block# = (0-words  $\times$  32) + offset. CPU bsf/ffs instructions give O(1) allocation.  
**RAM Bytes = Disk\_Size / (8 \* Block\_Size)**  
1.3 TB Disk with 512 B block = 302.68 MB RAM! (4KB block = 37.8 MB)
- Linked Free List:** Pointers inside free blocks. Zero space waste, but disastrous seek traversal.
- Grouping:** 1st free block holds (N-1) free block pointers + 1 pointer to next group block. 1 read yields N-1 blocks.
- Counting:** Stores tuples (start\_block, count) for contiguous extents; stored in B-Tree (ext4, XFS).

3. FILE PROTECTION & ACCESS MATRIX MODEL

**Access Matrix: ACL vs Capability List**

Theoretical 2D Matrix of Domains (Rows)  $\times$  Objects (Columns):

- Access Control Lists (ACL):** Decompose matrix by **Columns**. List attached to File (**F1  $\rightarrow [(UserA, rw), (UserB, r)]$** ).  
**Pros:** Trivial to revoke rights. **Cons:** Slower lookup on every open. Used in Windows NTFS, POSIX ACLs.
- Capability Lists (C-Lists):** Decompose matrix by **Rows**. Ticket/Token badge attached to Process (**ProcA  $\rightarrow [(F1, rw), (Printer, print)]$** ).  
**Pros:** O(1) instant verification. **Cons:** Extremely hard revocation. Used in seL4 microkernel.

**UNIX 9-Bit Octal Permissions (chmod 754)**

Divided into 3 classes: **Owner (u), Group (g), Others (o):**

**r = 4 (Read), w = 2 (Write), x = 1 (Execute)**

- chmod 754:**
  - Owner:  $4 + 2 + 1 = \mathbf{7}$  (rwx - full access)
  - Group:  $4 + 0 + 1 = \mathbf{5}$  (r-x - read & execute)
  - Others:  $4 + 0 + 0 = \mathbf{4}$  (r-- - read only)
- Special Modes:** SUID (4000: runs as owner, e.g. passwd), SGID (2000), Sticky Bit (1000: only owner can delete, e.g. /tmp 1777).

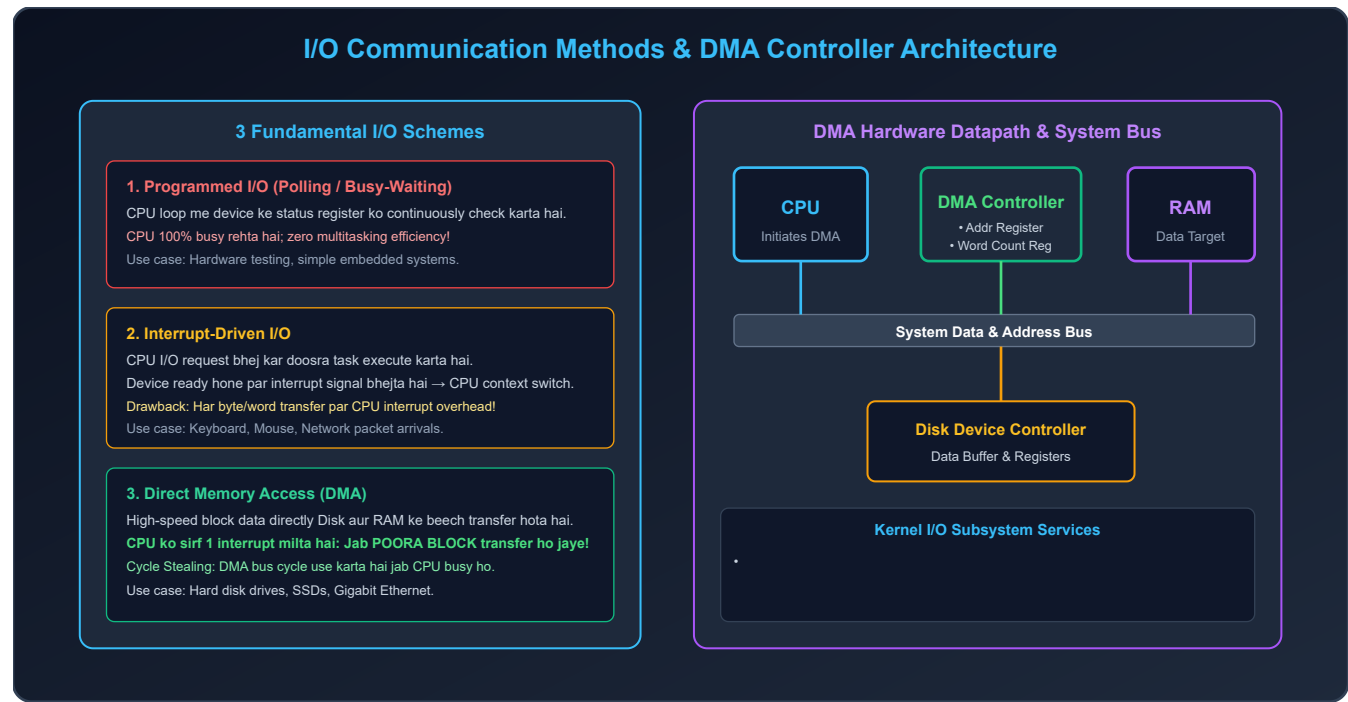
# Module 01: I/O Hardware and Kernel Subsystems

## 1. I/O Transfer Techniques

| Method                     | CPU Involvement              | Data Transfer Path          | Primary Use Case                        |
|----------------------------|------------------------------|-----------------------------|-----------------------------------------|
| Polling (Programmed)       | 100% Busy Waiting Loop       | Device → CPU Register → RAM | Simple dedicated microcontrollers       |
| Interrupt-Driven           | Interrupt on every byte/word | Device → CPU Register → RAM | Low-bandwidth devices (Keyboard, Mouse) |
| DMA (Direct Memory Access) | Single interrupt per BLOCK   | Device ↔ RAM (Direct bus)   | High-speed bulk storage (HDD, SSD, NIC) |

## 2. DMA Controller Architecture

CPU DMA controller me Base Address aur Word Count load karke release kar deta hai. DMA controller cycle stealing technique use karke high-speed data transfer execute karta hai.



**Buffering:** Speed mismatch handle karna (Single, Double, Circular) • **Caching:** Fast memory me frequent blocks rakhna • **Spooling:** Multi-user simultaneous print queue handle karna

## 3. Kernel I/O Subsystem Core Services

- **Buffering:** Speed mismatch aur peak load handle karna (Single, Double, Circular).
- **Caching:** Fast RAM me frequently accessed disk blocks maintain karna.
- **Spooling:** Exclusive devices (Printers) ke multi-user requests ko disk queue me serialize karna.

# Module 02: Disk Storage and Physical Architecture

## 1. Physical Anatomy & Units

Platters me concentric **Tracks** hote hain jo 512B/4KB ke **Sectors** me partitioned hote hain. Sabhi platters ke identical track radius ko vertical **Cylinder** kaha jata hai.

### Magnetic Hard Disk Physical Anatomy & Access Latency

#### Magnetic Disk Physical Structure

Spindle (Motor)

Actuator Arm

• Platter • Tracks (Concentric Rings) • Sectors

• Cylinder = Same track across all platters

#### Disk Access Time Formulation

##### 1. Seek Time ( $T_{seek}$ ) — LARGEST DELAY!

Read/Write head ko target track (cylinder) tak pahuchne ka samay.  
Mechanical arm movement: 3 - 10 ms (Dominant factor!).

##### 2. Rotational Latency ( $T_{rot}$ )

Desired sector ko ghoomkar head ke neeche aane ka samay.  
**Average Rotational Latency =  $1 / (2 \times \text{RPM}) \times 60 \text{ seconds}$**

##### 3. Transfer Time ( $T_{trans}$ )

Data ko disk surface se controller memory me transmit karne ka time.  
 $T_{trans} = (\text{Data Size}) / (\text{Transfer Rate})$

#### Total Disk Access Latency Formula

**$T_{access} = T_{seek} + T_{rot} + T_{trans} + T_{controller}$**

⇒ Goal of Disk Scheduling: Seek Time ( $T_{seek}$ ) minimize karna!

## 2. Latency Formulation & Solved Example

| Latency Component  | Mathematical Formula          | Example (5400 RPM, 8ms seek, 50MB/s)                  |
|--------------------|-------------------------------|-------------------------------------------------------|
| Seek Time          | Physical arm travel           | 8.00 ms (Dominant factor)                             |
| Rotational Latency | $30 / \text{RPM}$ (seconds)   | $30 / 5400 = 5.56 \text{ ms}$                         |
| Transfer Time      | Data Size / Transfer Rate     | $512 \text{ KB} / 50 \text{ MB/s} = 10.48 \text{ ms}$ |
| Total Latency      | Seek + Rot + Trans + Overhead | <b>24.24 ms</b>                                       |

# Module 03: Disk Scheduling — FCFS and SSTF

## 1. Algorithm Principles

- **FCFS:** Pure FIFO order. Fair with no starvation, but causes wild head swings.
- **SSTF:** Greedy local minimum search. Drastically reduces seek time, but causes **Starvation**.

### Disk Scheduling: FCFS vs SSTF Head Movement Comparison

Queue: 98, 183, 37, 122, 14, 124, 65, 67 | Initial Head Position = 53

#### 1. FCFS (First-Come, First-Served)

Sequence: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Calculations:

- $|98 - 53| = 45$
- $|183 - 98| = 85$
- $|37 - 183| = 146$  (Wild swing across disk!)
- $|122 - 37| = 85$
- $|14 - 122| = 108$
- $|124 - 14| = 110$
- $|65 - 124| = 59$
- $|67 - 65| = 2$

Total Head Movement = 640 Cylinders

- **Drawback:** Zero optimization; wild head swings.
- **Advantage:** Simple, fair, no starvation.

#### 2. SSTF (Shortest Seek Time First)

Sequence: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Calculations (Always pick nearest cylinder):

- $|65 - 53| = 12$
- $|67 - 65| = 2$
- $|37 - 67| = 30$
- $|14 - 37| = 23$
- $|98 - 14| = 84$
- $|122 - 98| = 24$
- $|124 - 122| = 2$
- $|183 - 124| = 59$

Total Head Movement = 236 Cylinders (63% Less!)

- **Serious Flaw:** Starvation of far requests!
- Greedy algorithm; not globally optimal.

## 2. Solved Numerical Summary

Queue: 98, 183, 37, 122, 14, 124, 65, 67 | Head = 53

- **FCFS Head Movement: 640 Cylinders**
- **SSTF Head Movement: 236 Cylinders** (63% reduction!)

# Module 04: Disk Scheduling — SCAN and C-SCAN

## 1. Comparison: SCAN vs C-SCAN

| Parameter          | SCAN (Elevator)                           | C-SCAN (Circular SCAN)                  |
|--------------------|-------------------------------------------|-----------------------------------------|
| Service Direction  | Both directions (Reverses at boundary)    | Strictly ONE direction only             |
| Return Path        | Services requests on way back             | Immediate fly-back to 0 without service |
| Wait Time Variance | Higher variance (favors middle cylinders) | Uniform wait time for all cylinders     |
| Boundary Touch     | Touches 0 or MAX cylinder                 | Touches both 0 and MAX cylinders        |

### Disk Scheduling: SCAN (Elevator) vs C-SCAN (Circular)

Queue: 98, 183, 37, 122, 14, 124, 65, 67 | Head = 53 | Direction: Towards 0

#### SCAN (Elevator Algorithm)

• Rule: Disk arm ek direction me aakhiri end (0) tak jata hai, phir reverse hokar doosre end ki taraf badhta hai.

Path: 53 → 37 → 14 → 0 → 65 → 67 → 98 → 122 → 124 → 183

Calculations:

- Downward to End 0: (53 - 0) = 53 tracks
- Reverse from 0 to 183: (183 - 0) = 183 tracks

Total Movement = 53 + 183 = 236 Cylinders

Total Head Movement = 236 Cylinders

• Advantage: No starvation! (Like an elevator)

• Inequity: Middle cylinders get serviced twice as often as boundary cylinders.

#### C-SCAN (Circular SCAN)

• Rule: Arm sirf EK DIRECTION me requests serve karta hai. End par pahunch kar sidha bina service kiye start par aata hai.

Path (Moving Up): 53 → ... → 199 → 0 → 14 → 37

Calculations (Upwards direction):

- 53 to 199: (199 - 53) = 146 tracks
- Fly-back 199 to 0: (199 - 0) = 199 tracks (No service)
- 0 to 37: (37 - 0) = 37 tracks

Total Movement = 146 + 199 + 37 = 382 Cylinders

• Advantage: Uniform waiting time for all cylinders!

• Treats cylinders as a continuous circular ring.

## 2. Solved Numerical Result

- SCAN (towards 0):  $(53 - 0) + (183 - 0) = 236$  Cylinders.
- C-SCAN (towards 199):  $(199 - 53) + (199 - 0) + (37 - 0) = 382$  Cylinders.

# Module 05: Disk Scheduling — LOOK and C-LOOK

## 1. LOOK vs C-LOOK Summary

| Algorithm | Boundary Behavior                   | Total Movement (Sample Queue) | Key Advantage                    |
|-----------|-------------------------------------|-------------------------------|----------------------------------|
| SCAN      | Always goes to extreme 0 or 199     | 236 cylinders                 | Standard elevator                |
| LOOK      | Reverses at last request            | 208 cylinders                 | Eliminates empty boundary travel |
| C-SCAN    | Fly-backs between 199 and 0         | 382 cylinders                 | Uniform wait time                |
| C-LOOK    | Fly-backs between max & min request | 322 cylinders                 | Uniform wait + 60 tracks saved   |

### Disk Scheduling: LOOK vs C-LOOK (Optimized Boundary Traversal)

Queue: 98, 183, 37, 122, 14, 124, 65, 67 | Head = 53 | Direction: Towards Lower Cylinders

#### LOOK Algorithm

- Optimization: Arm disk ke extreme end (0) tak NAHI jata! Sirf aakhiri pending request (14) tak jata hai, fir reverse hota hai.

Path: 53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

Calculations:

- Downward to lowest request (14):  $(53 - 14) = 39$  tracks
- Reverse from 14 up to 183:  $(183 - 14) = 169$  tracks

Total Movement = 39 + 169 = 208 Cylinders

Total Head Movement = 208 Cylinders

- SCAN (236) se 28 cylinders kam travel kiya!
- Unnecessary boundary track travel (0 to 14) eliminated.

#### C-LOOK (Circular LOOK)

- Optimization: 199 tak jane ke bajaye 183 par stop hota hai, aur 0 par jane ke bajaye directly 14 par jump karta hai!

Path (Moving Up): 53 → ... → 183 → 14 → 37

Calculations (Upwards direction):

- 53 to max request (183):  $(183 - 53) = 130$  tracks
- Jump from 183 to min request (14):  $(183 - 14) = 169$  tracks
- 14 to 37:  $(37 - 14) = 23$  tracks

Total Movement = 130 + 169 + 23 = 322 Cylinders

Total Movement = 130 + 169 + 23 = 322 Cylinders

- C-SCAN (382) se 60 cylinders kam travel kiya!
- Industry preferred practical circular disk scheduler.

## 2. Solved Numerical Output

- **LOOK (downward):**  $(53 - 14) + (183 - 14) = 208$  Cylinders.
- **C-LOOK (upward):**  $(183 - 53) + (183 - 14) + (37 - 14) = 322$  Cylinders.

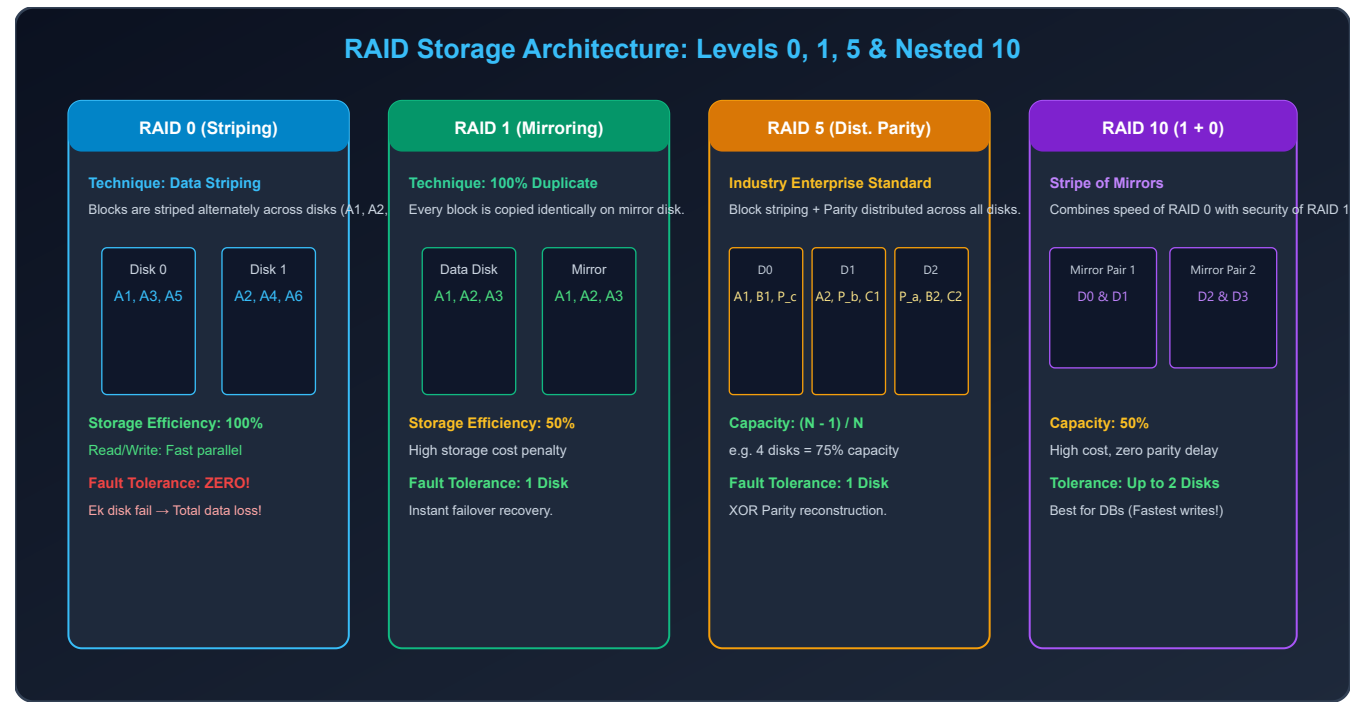
# Module 06: RAID Architecture — Levels 0 to 6

## 1. RAID Core Techniques

**Striping:** Parallel I/O speed • **Mirroring:** 100% duplicate copy • **Parity:** XOR mathematical recovery ( $D_1 \oplus D_2 = P$ ).

## 2. RAID Levels Comparison Table

| RAID Level | Technique          | Usable Capacity                   | Fault Tolerance         | Target Workload               |
|------------|--------------------|-----------------------------------|-------------------------|-------------------------------|
| RAID 0     | Block Striping     | $N \times \text{DiskSize}$ (100%) | 0 (Zero)                | High throughput, non-critical |
| RAID 1     | Mirroring          | 50% Capacity                      | 1 disk per pair         | OS boot disks, accounting     |
| RAID 5     | Distributed Parity | $(N - 1) \times \text{DiskSize}$  | 1 disk failure          | Standard enterprise servers   |
| RAID 6     | Dual Parity (P+Q)  | $(N - 2) \times \text{DiskSize}$  | 2 simultaneous failures | Critical cloud storage        |
| RAID 10    | Stripe of Mirrors  | 50% Capacity                      | Up to 2 disks           | High-performance Databases    |



## 3. Key Viva Question

**Why is RAID 10 preferred over RAID 01?**

RAID 10 me disk failure par sirf single mirror recover hota hai; do alag pairs se simultaneous disk failures survive ho sakte hain.

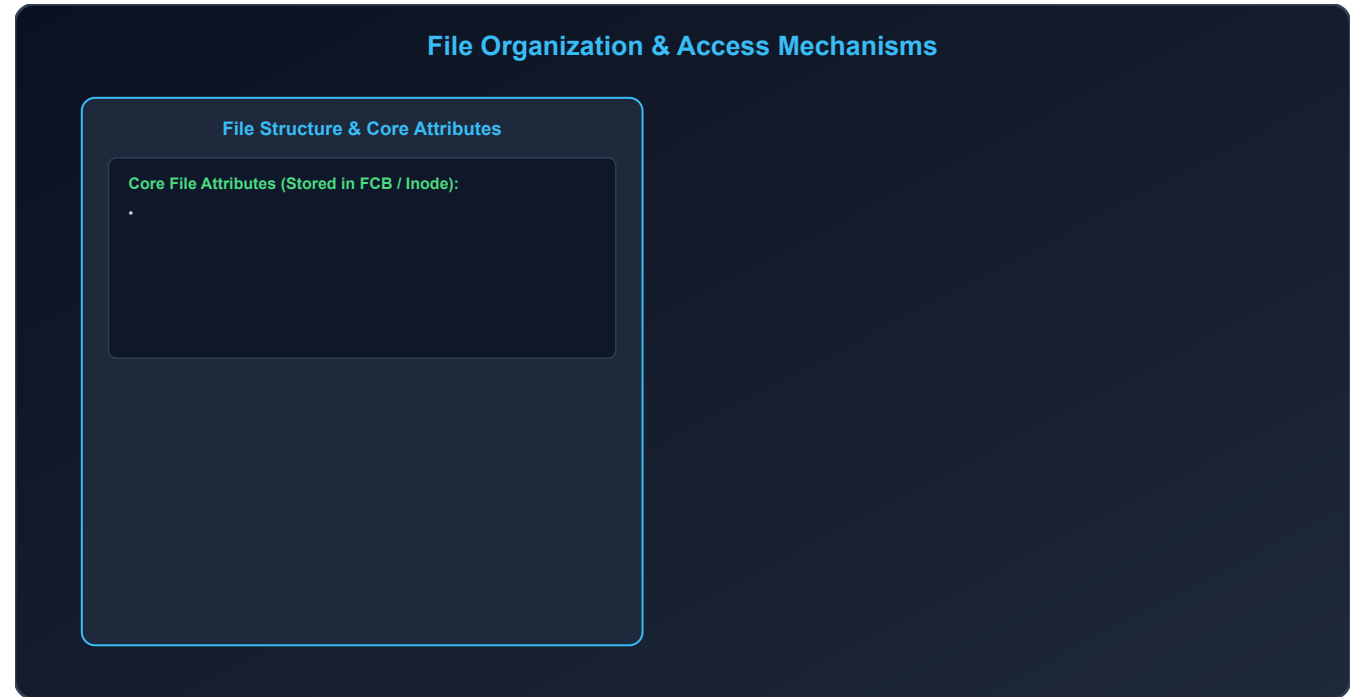
# Module 07: File Concept and Access Methods

## 1. Core File Attributes & Operations

Files OS ke logical storage units hain jinke metadata (Name, Identifier, Location, Size, Permissions) ko **File Control Block (FCB / Inode)** me store kiya jata hai.

## 2. Access Methods Comparison

| Method          | Mechanism                        | Random Access Speed  | Ideal Application         |
|-----------------|----------------------------------|----------------------|---------------------------|
| Sequential      | One byte/record after another    | Slow ( $O(n)$ )      | Compilers, media players  |
| Direct (Random) | Directly read block n via offset | Instant ( $O(1)$ )   | Relational Databases      |
| Indexed (ISAM)  | Index lookup → Direct fetch      | Fast ( $O(\log n)$ ) | Enterprise account lookup |



**Name:** Human-readable string (e.g., report.pdf) • **Identifier:** Unique integer tag (Inode number in Linux) • **Type:** Extension or magic bytes (.c, .txt, .exe) • **Location & Size:** Pointer to device & current byte size • **Protection & Timestamps:** Access rights, creation/mod time

**Standard File Operations:**

- `create()`: Allocate space & directory entry
- `write()` / `read()`: Transfer bytes at current file pointer
- `lseek()`: Reposition file pointer without I/O
- `truncate()`: Erase data keeping attributes intact
- `unlink()`: Release directory reference & free blocks

**3 File Access Methods**

1. **Sequential Access (Tape / Stream Model)** Information is processed in order, one record after another. Operations: `read_next()`, `write_next()`, `reset()`. Use case: Audio/video media streams, compilers scanning code.
2. **Direct / Random Access (Disk Block Model)** File is viewed as a numbered sequence of fixed-size blocks. Operations: `read(block_n)`, `write(block_n)`, `seek(position)`. Use case: Relational Databases, B-tree indexes, virtual disk images.
3. **Indexed Sequential Access Method (ISAM)** Built on top of direct access: Ek alag index table maintain hoti hai. Search index → Get block pointer →

Direct seek to record. Multi-level indexing used when index file itself becomes too large. Use case: Banking databases, enterprise flight reservation systems.

# Module 08: Directory Structures and File Sharing

## 1. Directory Structures Evolution

Single-Level (colliding) → Two-Level (MFD/UFD) → Tree-Structured (hierarchical) → Acyclic Graph (shared files via links).

## 2. Hard Link vs Soft Link (UNIX)

| Feature          | Hard Link                                  | Soft / Symbolic Link                           |
|------------------|--------------------------------------------|------------------------------------------------|
| Inode            | Same Inode number share karta hai          | Naya Inode create hota hai (contains pathname) |
| Target Deletion  | File intact rehti hai (until link_count=0) | <b>Dangling broken pointer</b> ban jata hai    |
| Cross-Filesystem | No                                         | <b>Yes</b>                                     |

Directory Organizational Structures & File Sharing

Directory Structural Models

1. Single-Level Directory

All files in ONE directory. Name collisions unavoidable!  
No grouping, poor scalability.

2. Two-Level Directory

Master File Directory (MFD) → User File Directory (UFD).  
Solves collision between users; but no internal subdirectories.

3. Tree-Structured Directory (Modern Standard)

Arbitrary depth subdirectories. Root (/) → Pathnames.  
Absolute path (/home/user/doc) vs Relative (/doc).

4. Acyclic-Graph Directory (Sharing)

Allows directories/files to have multiple parent paths!  
Hard links (Reference counting) & Soft links (Path shortcut).

Hard Link vs Symbolic Link & Dangling Pointer

Hard Link (ln target linkname):

• Directory entry sidha target file ke **\*\*SAME INODE\*\*** ko point karta hai.

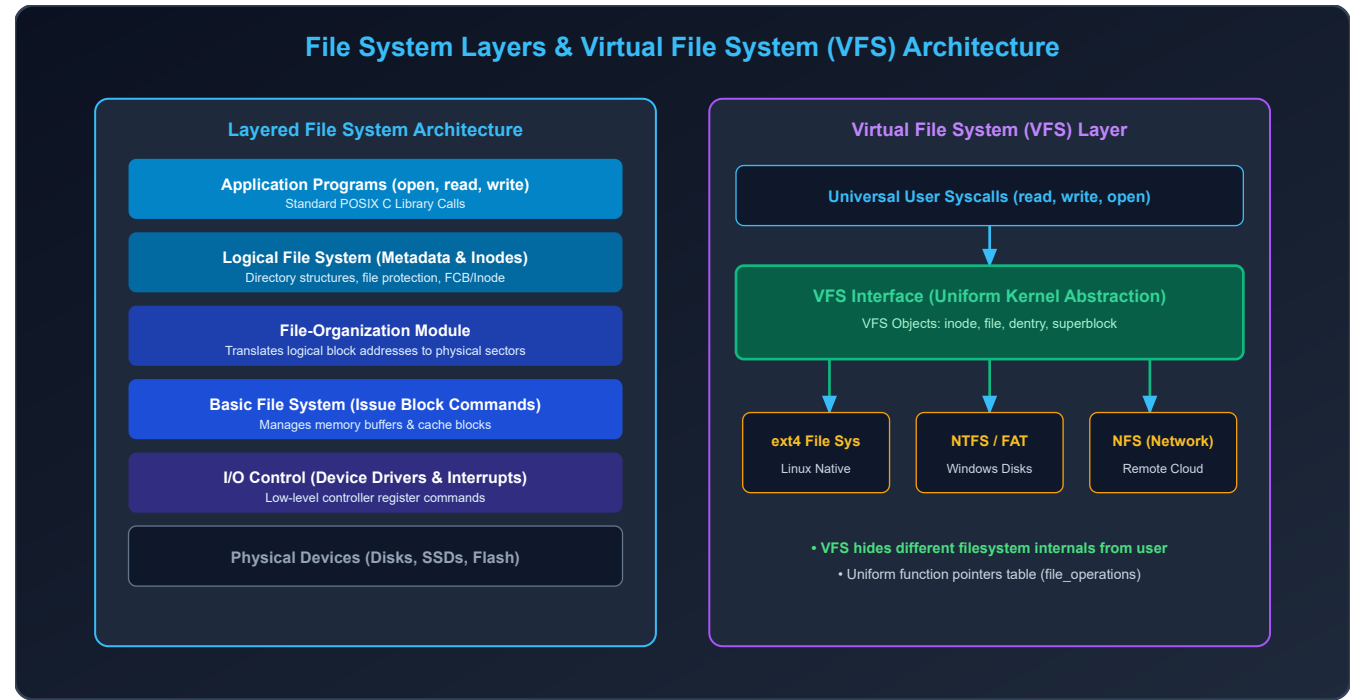
• Inode ka

link\_count increment ho jata hai (e.g., count = 2). • Original file delete karne par bhi file disk se delete nahi hoti! • File tabhi delete hoti hai jab link\_count == 0 ho jaye! • Limitation: Cannot cross filesystem boundaries; no directory hard links. Soft / Symbolic Link (ln -s target linkname): • Ek nayi file banti hai jiska apna ALAG INODE hota hai. • Is file ke andar target file ka pathname store hota hai. • Filesystems cross kar sakta hai aur directories par link ban sakta hai. • The Dangling Pointer Problem: Agar original file delete ho jaye, to soft link ek non-existent path ko point karta rehta hai ("Broken / Dangling Link")!

# Module 09: File System Implementation and VFS

## 1. On-Disk vs In-Memory Structures

| Structure              | Location             | Stored Information                            |
|------------------------|----------------------|-----------------------------------------------|
| Superblock             | Disk Partition       | Total blocks, free block count, inode list    |
| Inode / FCB            | Disk & Cached in RAM | File size, owner, permissions, block pointers |
| System-Wide Open Table | RAM                  | Global open files, reference count            |
| Per-Process Open Table | RAM (PCB)            | File descriptor (fd), current seek offset     |



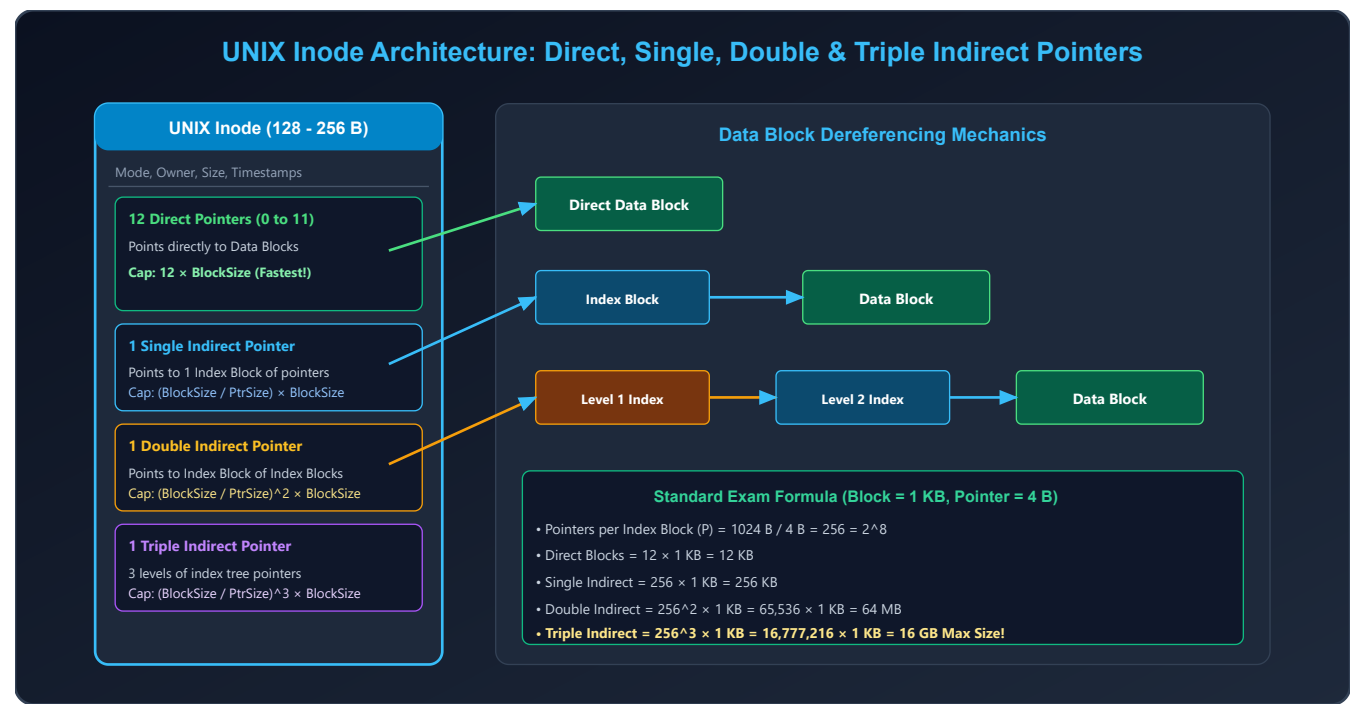
## 2. Virtual File System (VFS)

VFS application programs ko uniform POSIX API deta hai jabki internally ext4, NTFS, FAT, aur NFS ke driver implementations ko dynamically dispatch karta hai.

# Module 10: UNIX Inode Architecture and Calculations

## 1. 15-Pointer Hybrid Inode Model

UNIX Inodes combine 12 Direct pointers (for ultra-fast small file access) with 1 Single, 1 Double, and 1 Triple Indirect pointer (for huge multi-gigabyte file scaling).



# Module 11: File Allocation Methods

## 1. Allocation Strategies Overview

| Method     | Direct Access        | External Frag | File Expansion | Typical Implementation |
|------------|----------------------|---------------|----------------|------------------------|
| Contiguous | Instant ( $O(1)$ )   | Severe        | Very Difficult | CD-ROMs, swap space    |
| Linked     | Very Slow ( $O(n)$ ) | Zero          | Simple dynamic | FAT16 / FAT32          |
| Indexed    | Fast ( $O(1)$ )      | Zero          | Simple dynamic | UNIX Inodes, ext4      |

### File Allocation Methods: Contiguous, Linked & Indexed

#### 1. Contiguous Allocation

**Directory Entry:** <Start, Length>  
e.g. File "count" → Start: 0, Length: 4

Blk 0

Blk 1

Blk 2

Blk 3

**Advantages:**

- Fastest sequential & direct access
- Single seek to start of file

**Disadvantages:**

- External Fragmentation
- File growth problem (Cannot expand)
- Requires compaction

#### 2. Linked Allocation

**Directory Entry:** <Start, End>  
Blocks are scattered; contain next ptr

Blk 9 →

Blk 16 →

Blk 1 [-1]

**Advantages:**

- ZERO External Fragmentation
- Files can grow indefinitely

**Disadvantages:**

- Direct access extremely slow ( $O(n)$ )
- Pointer storage overhead (4B / block)
- Single corrupted pointer breaks file!

• FAT variant caches table in RAM

#### 3. Indexed Allocation

**Directory Entry:** <Index Block #>  
Dedicated block stores array of pointers

Index Block 19: [9, 16, 1, 10, 25]  
Direct pointers to scattered blocks

**Advantages:**

- Fast Direct Access without fragmentation
- No external fragmentation

**Disadvantages:**

- Index block overhead for tiny files
- Solved by Multi-level / Combined Inode

# Module 12: Free Space Management Techniques

Unit 5: I/O & File Systems

AKTU BCS401

Disk Management

## 1. Concept & Architectural Goals

Disk par files continuously create, modify aur delete hoti rehti hain. Operating System ko ye track rakhna padta hai ki kaun se blocks free hain aur kaun se allocated. Primary goals: **Fast Allocation**, **Minimal Memory Overhead**, and **Support for Contiguous Blocks**.



## 2. Core Free Space Management Techniques

### 2.1 Bit Vector / Bitmap

Har disk block ke liye 1 bit reserve hoti hai (0 = Allocated, 1 = Free). First free block calculate karne ka standard CPU hardware formula:

$\text{Block Number} = (\text{Number of 0-words} * 32) + \text{Offset of First 1-Bit}$

Modern CPUs me hardware instructions jaise bsf (bit scan forward) aur ffs (find first set) O(1) time me bit detect karte hain.

#### Bitmap RAM Overhead Derivation:

Agar Disk Size = 1.3 TB, Block Size = 512 B:

Total Blocks =  $1.3 \times 10^{12} / 512 = 2,539,062,500$  blocks.

Bitmap Size =  $2,539,062,500 \text{ bits} / 8 = \mathbf{302.68 \text{ MB RAM}}$ .

Agar Block Size 4 KB kar dein: Bitmap Size = **37.8 MB** (8x memory savings!).

### 2.2 Linked Free List vs Grouping vs Counting

| Parameter                | Bit Vector                   | Linked List                      | Grouping                         | Counting                |
|--------------------------|------------------------------|----------------------------------|----------------------------------|-------------------------|
| <b>RAM Overhead</b>      | High (Entire bitmap in RAM)  | Minimal (Head pointer)           | Low (Current group block)        | Low (B-Tree nodes)      |
| <b>Disk Overhead</b>     | Dedicated bitmap space       | Zero (Stored inside free blocks) | Zero (Stored inside free blocks) | Small (Extent table)    |
| <b>Contiguous Search</b> | Easy (Bit shift / AND masks) | Disastrous (High seek count)     | Poor                             | <b>Optimal (Native)</b> |
| <b>Modern Use</b>        | ext2/3, NTFS                 | Rare / Legacy                    | UNIX System V                    | ext4, XFS, ZFS          |

**Grouping:** Single free block stores (N-1) free block addresses + 1 pointer to next group block. 1 disk read provides hundreds of free block locations.

**Counting:** Stores tuples (start\_block, count) for contiguous extents. Drastically compresses allocation metadata.

# Module 13: File System Protection and Access Matrix

## 1. Concept & Access Matrix Model

Protection ensures unauthorized processes cannot perform prohibited operations (Read, Write, Execute, Delete) on files. The theoretical **Access Matrix** represents rights as a 2D table of Domains (Rows) × Objects (Columns).

### File System Protection & Access Control Matrix

Access Matrix Model, ACL vs Capability Lists, and UNIX 9-Bit Permissions

#### The Access Matrix Model (Domains x Objects)

| Domain \ Obj | File F1     | File F2           | Printer P1 |
|--------------|-------------|-------------------|------------|
| Domain D1    | read, write | -                 | print      |
| Domain D2    | read        | read, write, exec | -          |
| Domain D3    | -           | read              | print      |

Matrix is Sparse: Storing full 2D table wastes 95%+ space!

#### Decomposition: ACL vs Capability List

##### ACL (Column Slices)

Attached to OBJECT (File F1):

- Domain D1: [R, W]
- Domain D2: [R]
- Used by: NTFS, POSIX ACLs

##### Capability List (Row Slices)

Attached to DOMAIN (User/Proc D1):

- Token(F1): [R, W]
- Token(P1): [Print]
- Used by: Microkernels, Hydra

##### Comparison Summary:

- Revocation: Easy in ACL (modify file's list); Hard in C-List (find tokens).
- Access Check: Fast in C-List (ticket presented); Slow in ACL (search list).
- Least Privilege: C-List enforces principle of least privilege naturally.

#### UNIX / Linux 9-Bit File Protection Architecture (chmod 754)

OWNER (User 'u')

**r w x (4 + 2 + 1 = 7)**

Full Read, Write, Execute

GROUP ('g')

**r - x (4 + 0 + 1 = 5)**

Read and Execute Only

OTHERS ('o')

**r - - (4 + 0 + 0 = 4)**

Read Only Access

Resulting Octal Representation: `chmod 754 file.txt` | Special Bits: SUID (4000), SGID (2000), Sticky Bit (1000)

## 2. Implementation: ACL vs Capability Lists

| Metric       | Access Control List (ACL)                      | Capability List (C-List)                                   |
|--------------|------------------------------------------------|------------------------------------------------------------|
| Matrix Slice | Column Slices (Attached to File)               | Row Slices (Attached to Process/Domain)                    |
| Revocation   | <b>Very Easy</b> (Delete user from file's ACL) | <b>Hard</b> (Must trace and invalidate distributed tokens) |
| Access Check | Search ACL for matching user ID                | <b>O(1) Instant</b> (Process presents capability ticket)   |
| Adoption     | Windows NTFS, POSIX ACLs                       | Microkernels (seL4, Fuchsia)                               |

## 3. UNIX 9-Bit Protection Scheme (chmod)

UNIX simplifies matrix storage into 9 bits: 3 bits each for **User (Owner)**, **Group**, and **Others**.

Permissions: `r = 4, w = 2, x = 1`  
chmod 754: Owner = 4+2+1 (7), Group = 4+0+1 (5), Others = 4+0+0 (4)  
Terminal output: `-rwxr-xr--`

**Special Bits (12-bit mode):**

- **SUID (4000):** File executes with owner's privilege (e.g. passwd binary).
- **SGID (2000):** File executes with group privileges.
- **Sticky Bit (1000):** Used in /tmp (1777); only file creator can delete their file.

# Module 14: Unit 5 AKTU PYQs & Interview Cheatsheet

## 1. Disk Scheduling Benchmark & Formulas

### Unit 5 Master Benchmark & Formula Cheatsheet

Disk Scheduling Comparison, UNIX Inode Sizing & RAID Architecture Summary

#### Disk Scheduling Cylinder Movement (Benchmark)

Queue: 98, 183, 37, 122, 14, 124, 65, 67 | Head: 53 (0 - 199)

|        |                 |
|--------|-----------------|
| FCFS   | 640 cyl         |
| SSTF   | 236 cyl         |
| SCAN   | 236 cyl         |
| C-SCAN | 382 cyl         |
| LOOK   | 208 cyl (Best!) |
| C-LOOK | 322 cyl         |

**Key Exam Observation:**

LOOK eliminates unnecessary trips to disk boundary (0 and 199).

LOOK achieves highest throughput (208 cylinders vs FCFS 640 cylinders)!

#### Master Numerical Formulas (AKTU 10-Marks)

##### 1. Total Disk Access Time Formula

$T_{access} = T_{seek} + T_{rotational} + T_{transfer} + T_{controller}$

$T_{rotational} = 60 / (2 * RPM) \text{ seconds} = 30000 / RPM \text{ ms}$

For 7200 RPM: Avg Latency = 60 / 14400 = 4.167 ms

##### 2. UNIX Inode Max File Size Formula

$Entries/Block (K) = Block\_Size / Pointer\_Size$

$Max\ Size = (12 + K + K^2 + K^3) * Block\_Size$

For Block=4KB, Ptr=4B  $\rightarrow K = 1024$

$K^3 * 4KB = (2^{10})^3 * 2^{12} B = 2^{42} B = 4 \text{ TB limit!}$

##### 3. RAID Usable Capacity & Fault Tolerance

RAID 0: Capacity =  $N * S$  | Fault Tolerance = 0 disks

RAID 1: Capacity =  $S$  | Fault Tolerance =  $N - 1$  disks

RAID 5: Capacity =  $(N-1)*S$  | Fault Tolerance = 1 disk (Parity P)

##### 4. Bitmap Overhead

$Bitmap\_Bytes = Disk\_Size / (8 * Block\_Size)$

Increases with disk capacity; reduced 8x when block size quadrupled.

## 2. Solved 10-Mark Numerical: Disk Scheduling

Queue: 98, 183, 37, 122, 14, 124, 65, 67 | Initial Head: 53 | Range: 0 to 199

| Algorithm | Head Movement Trace Path                            | Total Seek Count      |
|-----------|-----------------------------------------------------|-----------------------|
| FCFS      | 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67       | 640 cylinders         |
| SSTF      | 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183       | 236 cylinders         |
| SCAN      | 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14 | 236 cylinders         |
| C-SCAN    | 53 → 199 → 0 → 14 → 37                              | 382 cylinders         |
| LOOK      | 53 → 183 (reverses at max request) → 14             | 208 cylinders (Best!) |
| C-LOOK    | 53 → 183 → 14 → 37                                  | 322 cylinders         |

## 3. Solved Numerical: UNIX Inode Capacity Derivation

**Problem:** Block Size = 4 KB, Pointer = 4 B. Inode: 12 Direct, 1 Single, 1 Double, 1 Triple Indirect.

- Pointers per block  $K = 4096 / 4 = 1024 \text{ pointers } (2^{10})$ .
- Direct =  $12 \times 4 \text{ KB} = 48 \text{ KB}$ .
- Single Indirect =  $1024 \times 4 \text{ KB} = 4 \text{ MB}$ .
- Double Indirect =  $(1024)^2 \times 4 \text{ KB} = 4 \text{ GB}$ .

5. Triple Indirect =  $(1024)^3 \times 4 \text{ KB} = \mathbf{4 \text{ TB}}$ .

**Total Max File Size  $\approx 4 \text{ TB}$ .**

## 4. Top Core Systems Interview Q&A

---

**Q: Why do SSDs not use SCAN or Elevator scheduling?**

A: SSDs use solid-state flash memory without mechanical moving heads or rotating platters. Rotational latency and seek time are 0. Every block has identical  $O(1)$  access latency. Hence, SCAN provides zero benefit and adds unnecessary scheduling overhead. Modern SSDs use NVMe FIFO / multi-queue schedulers.